

TaskForge: Movie Studio Edition

Design & Engineering Report - COS 214 Practical 4

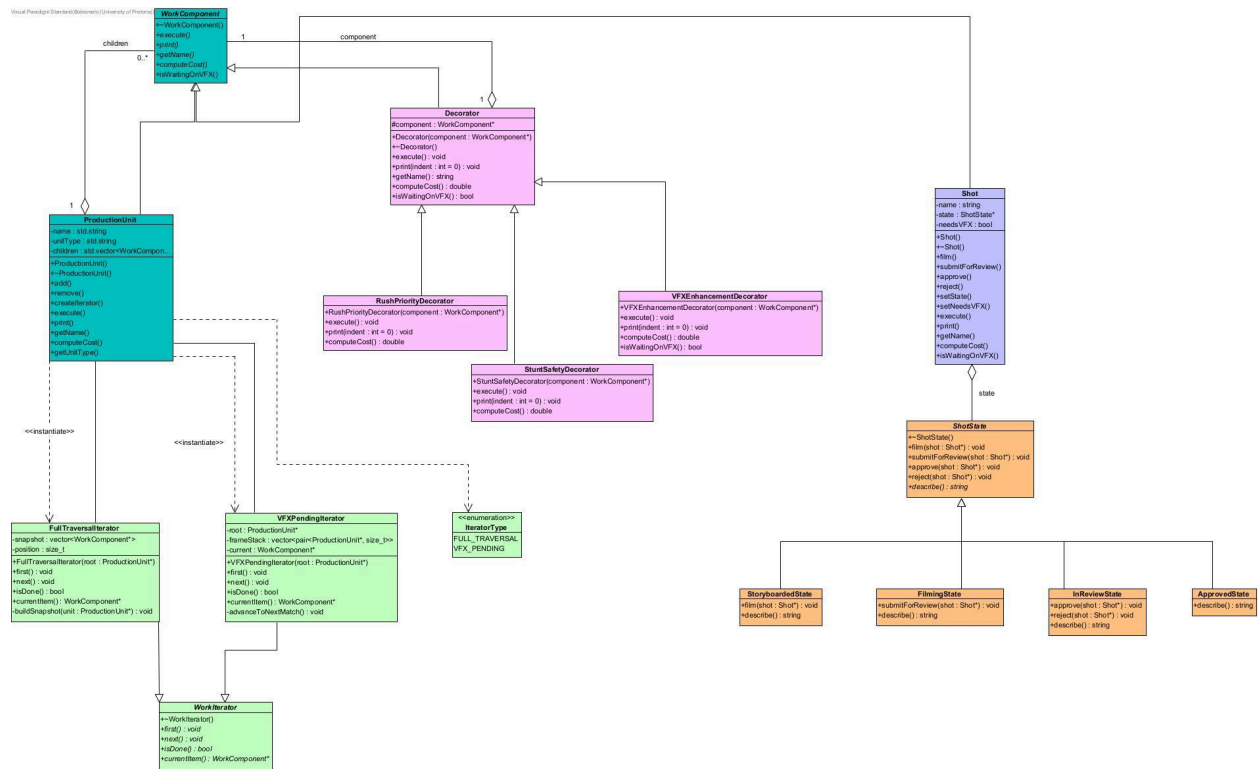
1. System Description and Team Details

Team Member	Student Number	Primary Pattern(s)
Dian le Roux	u25147065	Composite + Iterator
Jack Reid	u25265718	State
Boikemelo Masoka	u25128648	Decorator + Docker/Build

Domain: TaskForge models a movie studio's production hierarchy. A **Production** (the whole movie) is built from nested *production units* : departments, scenes, and sub-units, down to individual **shots**. A shot moves through a filming lifecycle, can be **decorated** with optional runtime responsibilities (rush priority, stunt safety checks, VFX enhancement), and the whole hierarchy can be traversed in two independent, purpose-specific ways.

Problem it solves: real productions need to treat a single shot and an entire department uniformly (Composite), walk the same hierarchy differently for different audiences -> a full script report vs. a VFX to-do list (Iterator) : track a shot's changing behaviour as it moves from storyboard to approval (State), and attach optional responsibilities to a shot at runtime without a subclass explosion (Decorator).

2. UML Class Diagram



3. GoF Participant Mapping

Pattern	Participants	Our Class
Composite	Component	WorkComponent
Composite	Leaf	Shot
Composite	Composite	ProductionUnit
Composite	Client	main.cpp
Iterator	Iterator	WorkIterator
Iterator	Concrete Iterator	FullTraversalliterator, VFXPendingIterator
Iterator	Aggregate / Concrete Aggregate	WorkComponent (createIterator hook) / ProductionUnit
State	Context	Shot
State	State	ShotState
State	Concrete State	StoryboardedState, FilmingState, InReviewState, ApprovedState
Decorator	Component	WorkComponent
Decorator	Decorator	Decorator
Decorator	Concrete Decorator	RushPriorityDecorator, StuntSafetyDecorator, VFXEnhancementDecorator
Decorator	Concrete Component	Shot

4. Design and Ownership Rationale

Composite

We use a single generic ProductionUnit class (rather than fixed Department/Scene/Production classes) so the hierarchy is a genuine recursive part-whole structure - a ProductionUnit can contain any WorkComponent, including another ProductionUnit, to arbitrary depth. A unitType string label ("Production", "Department", "Scene") gives each instance its domain meaning without needing a separate class per level. This satisfies the ≥ 3 -levels-below-root requirement without hardcoding depth.

Ownership: ProductionUnit owns every child in its children vector and deletes them in its destructor. remove() detaches without deleting, transferring ownership to the caller - this is what makes moving a Shot between scenes safe (detach, then re-add elsewhere, with no point where it's unowned or double owned).

Iterator

Two concrete iterators, deliberately using different access strategies from the lecture notes on accessing an aggregate's elements:

- **FullTraverseIterator** : a **snapshot iterator**. Copies pointers into a vector at construction via a recursive depth-first walk, giving a stable traversal for a "full script report" that won't change mid-read.
- **VFXPendingIterator** : a **live iterator**. Holds a stack of (unit, next-child-index) frames and reads each ProductionUnit's children fresh on every step, so a shot flagged for VFX after construction is still picked up. Appropriate for a to-do list that should reflect the current state of the world.
-

Access to the aggregate's internals: both concrete iterators are declared friend classes of ProductionUnit, and nothing else is. This is what lets them read the private children container directly without ProductionUnit exposing a public **getter**: which would let client code (main.cpp) bypass the pattern entirely.

Neither iterator owns what it points to. The WorkComponent pointers they hold are borrowed references into the live tree; only ProductionUnit (and Decorator, for its wrapped component) ever calls delete on them.

State

Why shot delegates to a ShotState pointer: The reason why the Shot class delegates its life cycle to a shotpointer is because this allows it to change depending on its current state without having to put state specific logic in the shot class.

how invalid transitions are handled by default in the base ShotState class: invalid transitions are handled by the base shot class by rejecting the operation and leaving and by leaving the shot state unchanged. Each concrete state only implements the transitions that are valid for its state.

the reshoot loop from InReview back to Filming; ownership of the state object: When a shot is rejected during review, it transitions from InReview back to Filming, creating a

reshootloop. After the shot is filmed again it can be submitted for review again. If it is approved it transitions to Approved.

Decorator

Why wrap WorkComponent instead of subclassing Shot directly?

The Decorator classes wrap the shared WorkComponent interface instead of subclassing Shot directly. This avoids needing a separate class for every possible combination of RushPriority, StuntSafety, VFXEnhancement. This allows wrapping decorators around decorators without touching Shot's own code.

How stacking actually works

Each concrete decorator both implements and wraps a WorkComponent, so decorators can be stacked in any order, with each layer doing its own job before passing the call down to the one it wraps. This creates a chain like this RushPriorityDecorator -> StuntSafetyDecorator -> VFXEnhancementDecorator -> Shot. The decorated object still works anywhere a plain Shot would, since it keeps the same interface.

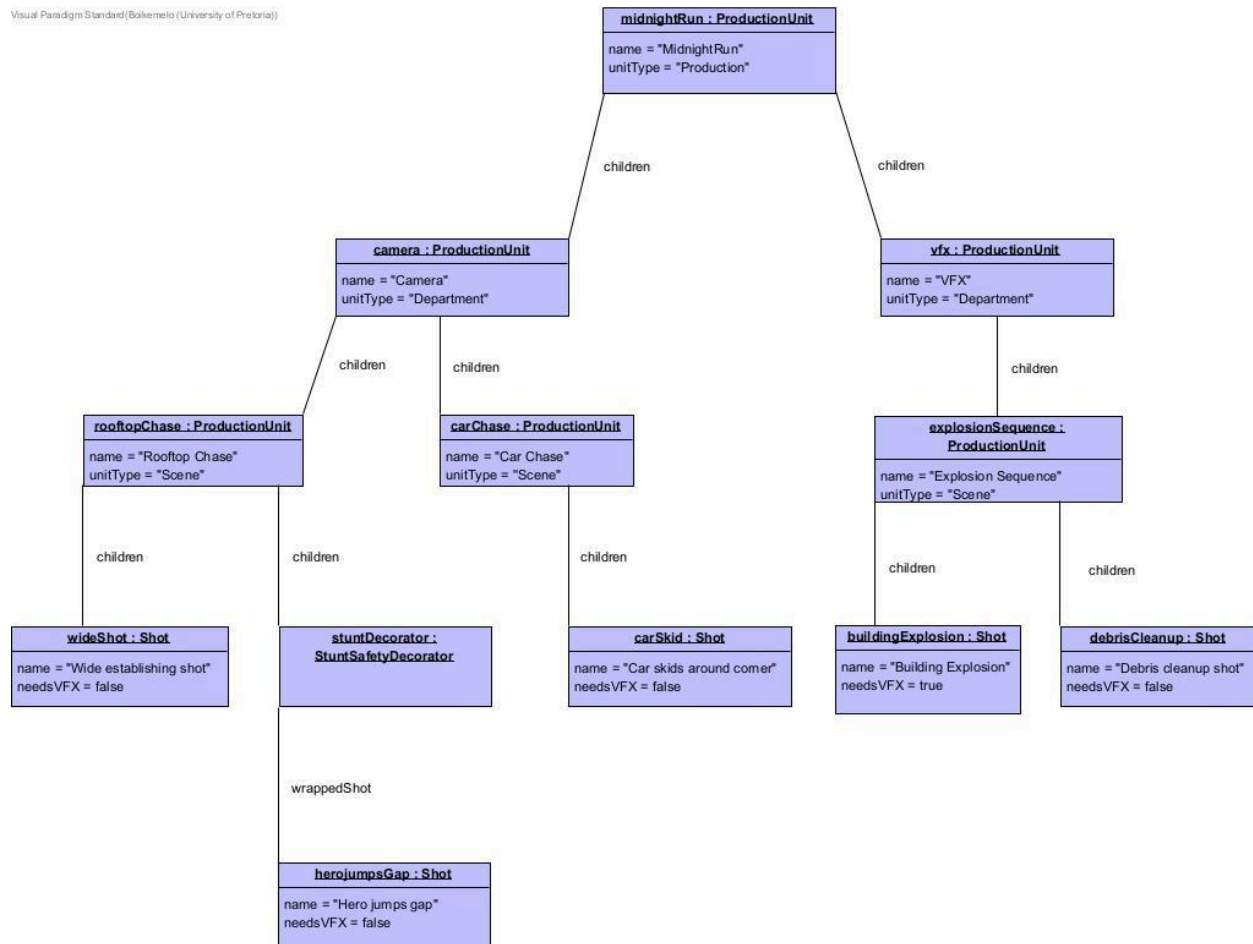
Ownerships and deletion:

Every abstract decorator base class owns what it wraps. Its destructor deletes the component it wraps. Deleting the outermost decorator triggers a chain reaction that cleans up every layer underneath it with no memory leaks and no manual cleanup needed

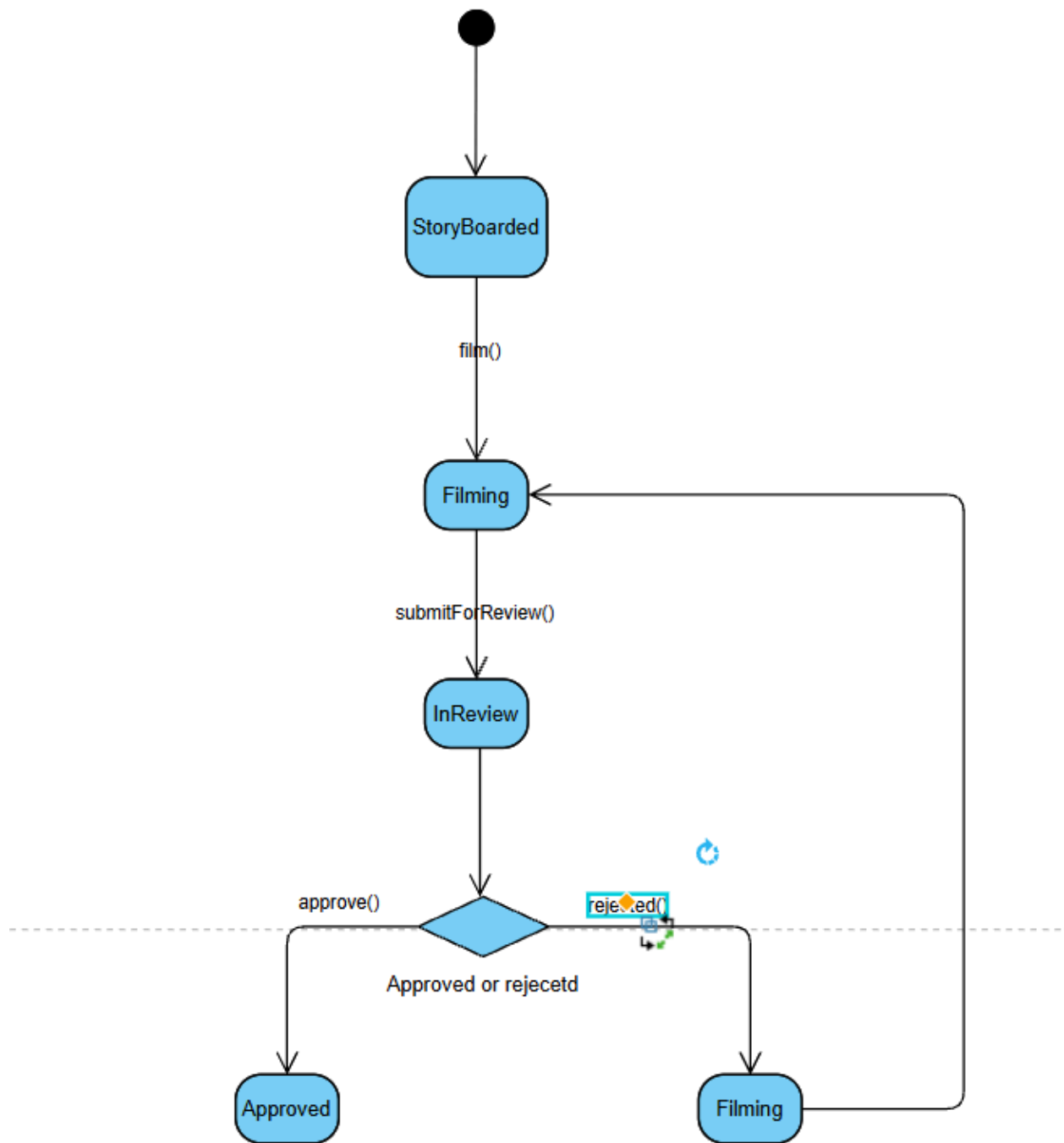
5. Object Diagram

This object diagram shows a real runtime snapshot of the TaskForge hierarchy: one Production containing two Department units, three scene units and five shot leaves. One shot “Hero jumps gap” is wrapped in a StuntSafetyDecorator demonstrating that a decorated shot slots into the Composite tree exactly like a plain one. Links are labelled with the actual member variable each one represents

Visual Paradigm Standard (Bokemelo (University of Pretoria))



6. State Diagram



7. Activity Diagrams

Diagram 1: traversal made visible: "Generate full script report" -> show the iterator's

first()/next()/isDone() steps as explicit actions in the flow (not hidden behind one "process all" box), including a loop.

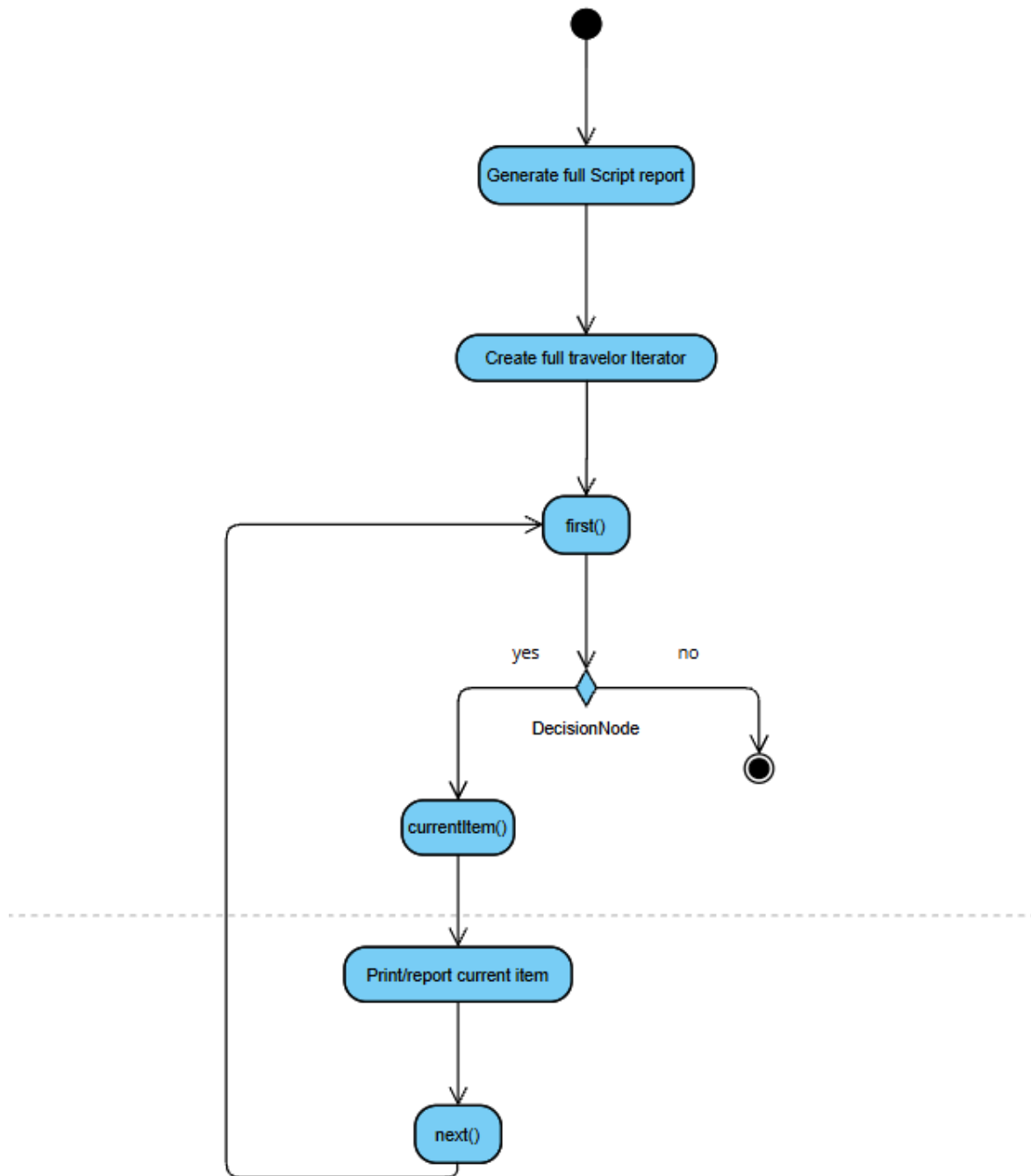


Diagram 2: lifecycle/config conditional: "Submit shot for review" -> decision node on the shot's current state, guard conditions, a merge, and the reshoot loop back to Filming.

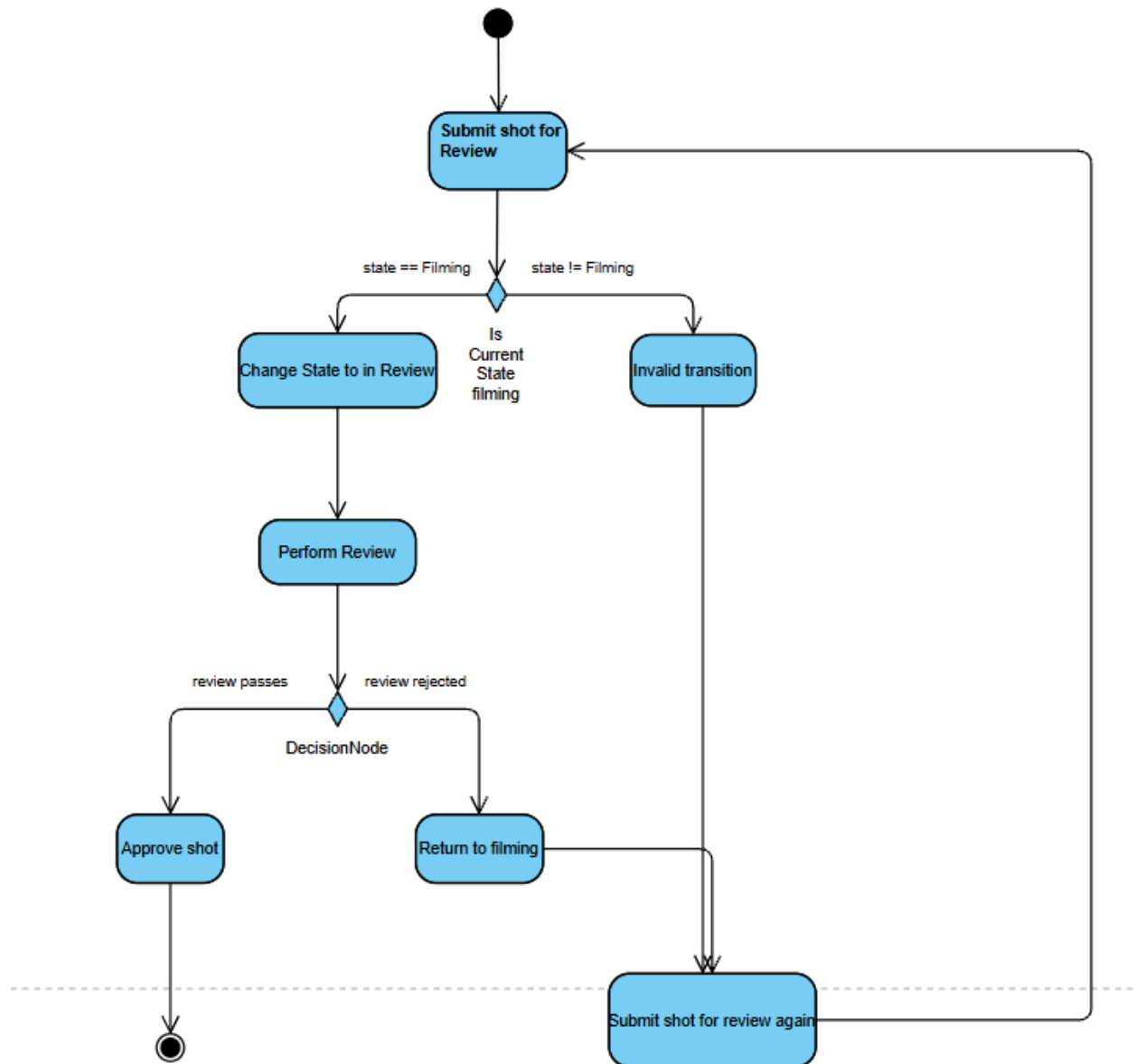
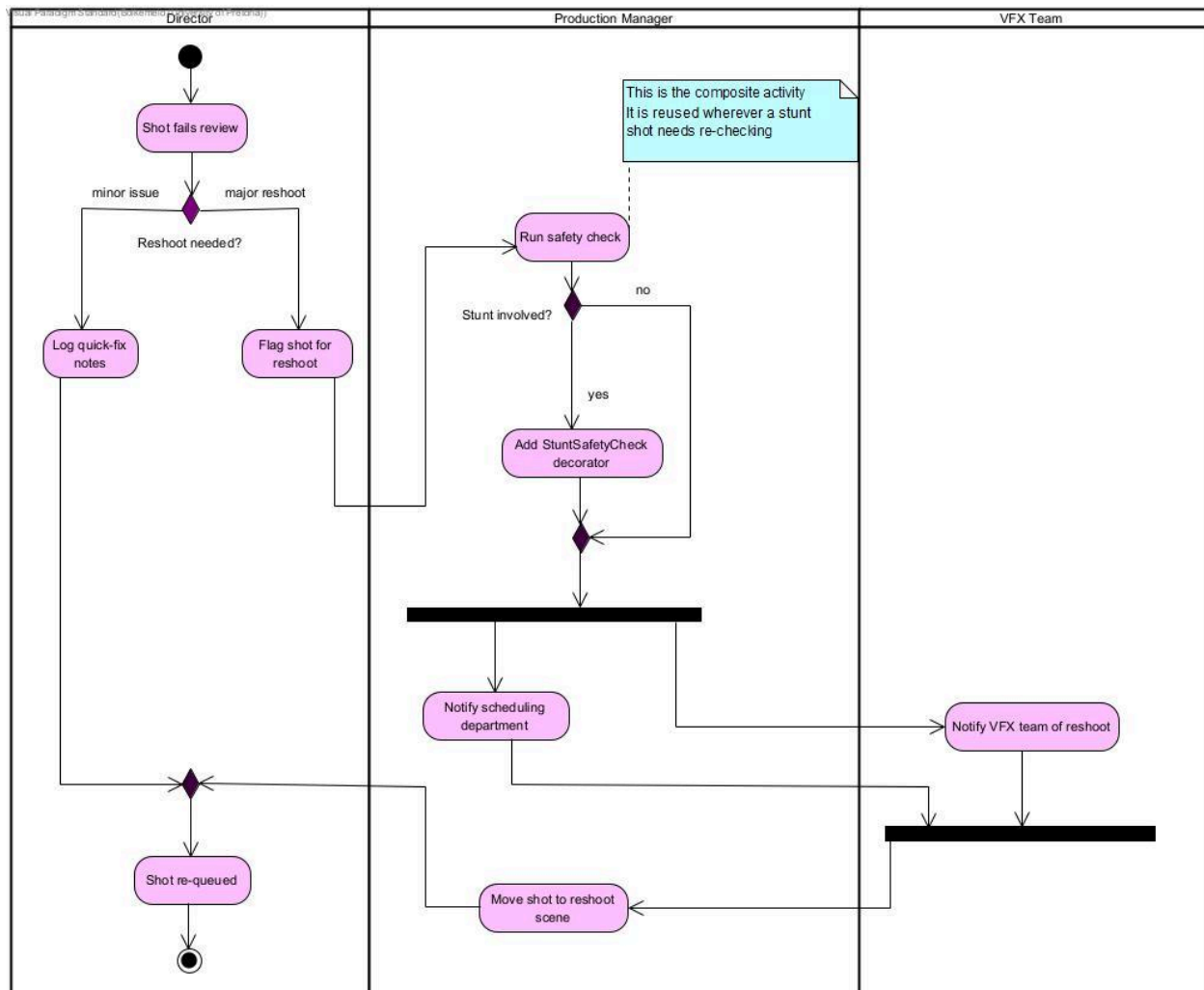


Diagram 3: distinctive workflow: "Handle reshoot crisis" → swimlanes (Director / VFX team / Production manager), a fork/join (parallel department notification), and a called/composite

activity (ex: "Run safety check" reused as a sub-activity).



8. Traversal-Invalidation Policy and Other Key Design Decisions

Ownership / destruction summary:

- ProductionUnit owns and deletes its children.
- Decorator owns and deletes the WorkComponent it wraps.
- Shot owns and deletes its ShotState. [TODO: confirm once State is implemented]
- main.cpp owns only the root ProductionUnit; deleting it cascades through the whole tree.

Traversal-in-progress policy (deliberate asymmetry):

- FullTraversalIterator snapshots at creation: a structural change mid-walk (ex: a reshoot) won't appear until the next report is generated. Appropriate for a report that shouldn't rewrite itself while being read.
- VFXPendingIterator is live: a shot that suddenly needs VFX appears on the to-do list immediately.
- **Safety rule:** a Shot may be moved between units while a live iterator is active, but may not be deleted from the tree while any live iterator references it.

Full console output from both demo scenarios, confirming correct behaviour across all four patterns in a single run:

```
TaskForge -- Movie Studio Edition
```

```
===== Scenario A: Daily production meeting =====
-- Nested tree view (ProductionUnit::print(), called once on the
root) --
Production: Midnight Run
  Department: Camera
    Scene: Rooftop Chase
      Shot: Wide establishing shot [Storyboarded]
        [STUNT SAFETY]
        [RUSH PRIORITY]
      Shot: Hero jumps gap [Storyboarded]
  Department: VFX
    Scene: Explosion Sequence
      Shot: Building explosion [Storyboarded] (VFX pending)
      Shot: Debris cleanup shot [Storyboarded]

-- Flat full script report (FullTraversalIterator - every node, once
each) --
- Camera
- Rooftop Chase
- Wide establishing shot
- Hero jumps gap
- VFX
```

- Explosion Sequence
- Building explosion
- Debris cleanup shot

Total estimated cost so far: 4500

-- Driving 'Hero jumps gap' through its lifecycle --

-- Attempting an invalid transition (approve() while still Storyboarded) --

Invalid transition: approve() is not allowed from this state.

(press Enter to continue)

===== Scenario B: Reshoot crisis =====

-- Debris cleanup shot goes through review and fails (reshoot loop)

--

Shot: Debris cleanup shot [Filming]

-- Two independent iterators over the SAME tree, interleaved --

VFX-pending iterator, first match: Building explosion

Full-traversal iterator, first item: Camera

-- Mid-crisis: debris shot now ALSO needs VFX cleanup work --

VFX-pending iterator, next match (should now include debris shot):

Debris cleanup shot

-- Structural change: moving 'Debris cleanup shot' to a different scene --

-- Structural change: a brand-new shot is added AFTER freshReportIt's snapshot was taken --

freshReportIt keeps walking its ORIGINAL snapshot - the new shot will NOT appear:

- Rooftop Chase
- Wide establishing shot
- Hero jumps gap
- VFX
- Explosion Sequence
- Building explosion
- Debris cleanup shot

A BRAND NEW FullTraversalIterator sees the current, updated tree instead:

- Camera
- Rooftop Chase
- Wide establishing shot

- Hero jumps gap
- Debris cleanup shot
- VFX
- Explosion Sequence
- Building explosion
- Pickup insert shot

(press Enter to continue)

Shutting down - deleting the production tree cascades through everything it owns.

- The recursive tree view and flat iterator report both showing correctly, side by side, with no duplication.
 - Total estimated cost so far: 4500 —> our proof that `computeCost()` correctly sums through the composite tree and the decorator stack (500 base + 2000 rush + 500 stunt + others = the aggregate).
 - The invalid transition rejection message appearing exactly where expected.
 - The reshoot loop (Debris cleanup shot [Filming] after `reject()`).
 - The two independent iterators genuinely interleaved (VFX-pending iterator, first match: Building explosion followed immediately by Full-traversal iterator, first item: Camera -> two different positions, two different traversals, same tree, at the same time).
 - The live vs. snapshot proof in one screen: `freshReportIt` still shows the pre move state and never picks up Pickup insert shot, while the brand new iterator right after it shows the fully updated tree
-

9. GDB Bug Investigation

```
(gdb) break RushPriorityDecorator::print
Breakpoint 1 at 0x24c8: file src/decorator/RushPriorityDecorator.cpp,
line 14.
(gdb) run
Starting program: /taskforge/taskforge
TaskForge -- Movie Studio Edition
```

```
===== Scenario A: Daily production meeting =====
-- Nested tree view (ProductionUnit::print(), called once on the
root) --
Production: Midnight Run
  Department: Camera
    Scene: Rooftop Chase
      Shot: Wide establishing shot [Storyboarded]
        [STUNT SAFETY]
```

```
Breakpoint 1, RushPriorityDecorator::print (this=0xaaaadb8904f0,
indent=3) at src/decorator/RushPriorityDecorator.cpp:14
14      void RushPriorityDecorator::print(int indent) const {
(gdb) print indent
$1 = 3
(gdb) next
16          std::cout << std::string(static_cast<size_t>(indent) * 2,
' ') << std::endl;
(gdb) next

17          Decorator::print(indent);
(gdb) list
12      }
13
14      void RushPriorityDecorator::print(int indent) const {
15
16          std::cout << std::string(static_cast<size_t>(indent) * 2,
' ') << std::endl;
17          Decorator::print(indent);
18
19      }
20
21      double RushPriorityDecorator::computeCost() const {
```

----- After Fix:

```
Production: Midnight Run
  Department: Camera
```

Scene: Rooftop Chase

Shot: Wide establishing shot [Storyboarded]

[STUNT SAFETY]

[RUSH PRIORITY]

Shot: Hero jumps gap [Storyboarded]

[RUSH PRIORITY] now appears at the correct indent, immediately below [STUNT SAFETY], matching how its sibling decorators format their own tags

10. Valgrind Evidence

Valgrind evidence — final application, run inside Docker:

```
$ valgrind --leak-check=full --show-leak-kinds=all ./taskforge <
/dev/null
==15== Memcheck, a memory error detector
==15== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et
al.
==15== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright
info
==15== Command: ./taskforge
==15==
[... full program output from both demo scenarios ...]
==15==
==15== HEAP SUMMARY:
==15==      in use at exit: 0 bytes in 0 blocks
==15==    total heap usage: 84 allocs, 84 frees, 80,195 bytes
allocated
==15==
==15== All heap blocks were freed -- no leaks are possible
==15==
==15== For lists of detected and suppressed errors, rerun with: -s
==15== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
(press Enter to continue)

Shutting down - deleting the production tree cascades through everything it owns.
==15==
==15== HEAP SUMMARY:
==15==      in use at exit: 0 bytes in 0 blocks
==15==    total heap usage: 84 allocs, 84 frees, 80,195 bytes allocated
==15==
==15== All heap blocks were freed -- no leaks are possible
==15==
==15== For lists of detected and suppressed errors, rerun with: -s
==15== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
root@4228a871c6c9:/taskforge#
```

Result: 84 allocations, 84 frees -> zero bytes leaked, zero errors. Confirms the ownership policy documented in Section 4/8 (ProductionUnit deletes its children, Decorator cascades through the wrapped chain, Shot deletes its ShotState on every transition) holds correctly across both full demo scenarios, including the reshoot loop's repeated state swaps and the structural move/add

operations in Scenario B.

11. GitHub Repository and Workflow Reflection

Repository: https://github.com/BitwizeBandit/COS214_Prac_4_Group_1

Work was split along the four required patterns, with each person owning one area end to end rather than mixing contributions across files. Dian set up the project's Docker and Makefile build environment early on, then implemented the Composite pattern (WorkComponent, ProductionUnit) and the Iterator pattern (WorkIterator and both concrete iterators), later adding Shot as the shared leaf class linking Composite and State. JR implemented the State pattern — u25147065ShotState and the four concrete lifecycle states. Kemi implemented the Decorator pattern in full, working through the abstract Decorator class and header structure before implementing the three concrete decorators.

Decorator and State were developed in parallel on separate branches and brought together through a pull request rather than a direct merge to main, which is also where an early integration bug surfaced: some logic had gotten mixed up between two of the concrete decorator classes during development, and Kemi caught and corrected it before it made it any further. A similar real iteration happened on the State side — a stray test file was accidentally added to the repository and an attempted cleanup only partially removed it, which was only caught later during a full codebase audit, not at the time.

Once all three pattern areas existed, the final integration work — building the two demonstration scenarios in main.cpp that actually exercise all four patterns together, and working through the ownership rules so Composite, Decorator, and State could all safely share the same tree — was done last, alongside a bug found through an actual GDB debugging session in one of the decorator classes, which is documented in Section 9.

12. Team Contribution Statement

Team Member	Pattern / Area Owned	Contribution
Dian Le Roux	Composite (WorkComponent, ProductionUnit) and Iterator (WorkIterator, both concrete iterators); Docker/Makefile setup; final main.cpp integration	The friend-based access rule, snapshot vs. live traversal, the ownership/destruction chain, the GDB investigation
Jack Reid	State (ShotState and the four concrete states)	The lifecycle transitions, the reshoot loop, how invalid transitions are rejected
Boikemelo Masoka	Decorator (Decorator and the three concrete decorators)	How decorators stack and delegate, the cost/behaviour chaining, the bug fixed in RushPriorityDecorator

All three of us reviewed each other's pattern implementations during a full codebase audit before submission, and we can discuss areas beyond our own primary contribution